

Compiler Construction

~ Instruction Scheduling ~

Table of Contents

- 1 Introduction
- 2 Data dependencies
- 3 Instruction dependencies

Why and When reordering?

Goal

Reorder the instructions within each basic block

- preserves the dependencies between those instructions (and hence the correctness of the program)
- obtains the best performances

A word on Dependencies 1/4

Two instructions are **independent** if they can be permuted without altering the consistency

A word on Dependencies 2/4

- The 3 following instructions are independent

$\text{inst}_1 : a \leftarrow 42$

$\text{inst}_2 : b \leftarrow 51$

$\text{inst}_3 : c \leftarrow 0$

- inst_1 , inst_2 and inst_3 can then be reordered

$\text{inst}_1 : a \leftarrow 42$	$\parallel$	$\text{inst}_1 : a \leftarrow 42$	$\parallel$	$\text{inst}_3 : c \leftarrow 0$
$\text{inst}_2 : b \leftarrow 51$	$\parallel$	$\text{inst}_3 : c \leftarrow 0$	$\parallel$	$\text{inst}_1 : a \leftarrow 42$
$\text{inst}_3 : c \leftarrow 0$	$\parallel$	$\text{inst}_2 : b \leftarrow 51$	$\parallel$	$\text{inst}_2 : b \leftarrow 51$

$\text{inst}_1 : c \leftarrow 0$	$\parallel$	$\text{inst}_1 : b \leftarrow 51$	$\parallel$	$\text{inst}_3 : b \leftarrow 51$
$\text{inst}_2 : b \leftarrow 51$	$\parallel$	$\text{inst}_3 : c \leftarrow 0$	$\parallel$	$\text{inst}_1 : a \leftarrow 42$
$\text{inst}_3 : a \leftarrow 42$	$\parallel$	$\text{inst}_2 : a \leftarrow 42$	$\parallel$	$\text{inst}_2 : c \leftarrow 0$

A word on Dependencies 3/4

Two instructions are **dependant** if the first one needs to be executed before the second one.

A word on Dependencies 4/4

- The 3 following instructions are dependent, i.e. **no reordering is possible!**

$\text{inst}_1 : a \leftarrow 42$

$\text{inst}_2 : b \leftarrow a + 51$

$\text{inst}_3 : c \leftarrow b \times 12$

- Two kinds of dependencies:
 - ▶ **Data dependencies:** if one instruction uses a "variable" computed by another instruction.
 - ▶ **Instruction dependencies:** if one instruction is a "cjump", the instruction right after it depends on it.

Table of Contents

- 1 Introduction
- 2 Data dependencies
- 3 Instruction dependencies

Read after Write (RAW)

An instruction reads from a location (variable or memory) after an earlier instruction has written to it.

```
inst1 : lw    $2, 0($4)
inst2 : addi  $6, $2, 42
```

inst₁ and inst₂ cannot be permuted, otherwise inst₂ would read an old value from \$2.

Write after Read (WAR)

An instruction writes to a location after an earlier instruction has read from it.

```
inst1 : lw    $2, 0($4)
inst2 : addi  $4, $12, 42
```

inst₁ and inst₂ cannot be permuted, otherwise inst₁ would read a new value for \$4

Write after Write (WAW)

An instruction writes to a location after an earlier instruction has written to it.

inst₁ : add \$1, \$2, \$3

inst₂ : add \$1, \$5, \$6

inst₁ and inst₂ cannot be permuted, otherwise inst₁ would write an old value in \$1

CPU Execution and Pipelining

- **Sequential execution**

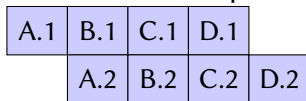
- ▶ CPU completes one instruction before starting the next



- ▶  time

- **Pipelined (split execution)**

- ▶ Instructions overlap



- ▶  time

Instructions Pipeline

The microprocessor (MIPS) contains 5 stages:

- **IF**: Instruction Fetch
- **ID**: Instruction Decode.
- **EX**: Execute instructions requiring the ALU
- **ME**: Read/write into Memory
- **WB**: Write Back. Results are written into registers.

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆	cycle ₇	cycle ₈	cycle ₉
instr ₁	IF	ID	EX	ME	WB				
instr ₂		IF	ID	EX	ME	WB			
instr ₃			IF	ID	EX	ME	WB		
instr ₄				IF	ID	EX	ME	WB	
instr ₅					IF	ID	EX	ME	WB

Stages in Action - 1

Assembly: `lw $2 0($4)`

- **IF:** The Program Counter (PC) holds the address of the next instruction.
 - ▶ Instruction at address PC loaded into Current Instruction Register (CIR)
 - ▶ PC increments to next instruction
- **ID:** CIR holds: 100011 00100 00010 0000000000000000
 - ▶ 100011 opcode (lw)
 - ▶ 00100 rs = \$4 (base register)
 - ▶ 00010 rt = \$2 (destination register)
 - ▶ 0 immediate offset
- **EX:** ALU computes memory address: $1000 + 0 = 1000$
- **ME:** $\text{Memory}[1000] = 55$
- **WB:** Result written back to register file: $\$2 = 55$

Stages in Action - 2

Assembly: `addi $5 $2 10`

- **IF:** The Program Counter (PC) holds the address of the next instruction.
 - ▶ Instruction at address PC loaded into Current Instruction Register (CIR)
 - ▶ PC increments to next instruction
- **ID:** CIR holds: 001000 00010 00101 0000000000001010
 - ▶ 001000 opcode (`addi`)
 - ▶ 00010 `rs` = `$2`
 - ▶ 00101 `rt` = `$5`
 - ▶ 10 immediate value
- **EX:** ALU performs the operation: $10 + 42 = 52$
- **ME:** none
- **WB:** Result written back to register file: `$5` = 52

Hazard: RAW dependencies 1/4

Some instruction requires a result computed by a previous one!

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆	cycle ₇	cycle ₈
lw \$2, 0(\$4)	IF	ID	EX	ME	WB			
addi \$5, \$2, 10		IF	•	•	•	ID	EX	ME

- lw produces its result into \$2 at the **end** of wb (cycle 5)
- addi needs \$2 at the **start** of ID (register read)
- 3 stalls (cycles 3, 4, 5) $\Rightarrow$ addi's ID must wait until wb completes

Hazard: RAW dependencies 2/4 - with split-cycle

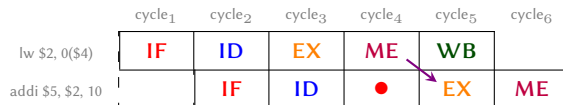
Assumption: writes in the *first half* and reads in the *second half* of the **same** cycle.

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆	cycle ₇
lw \$2, 0(\$4)	IF	ID	EX	ME	WB		
addi \$5, \$2, 10		IF	•	•	ID	EX	ME

- lw's WB writes \$2 in the **first half** of cycle 5
- addi's ID reads \$2 in the **second half** of cycle 5 $\Rightarrow$ same cycle is safe
- 2 stalls (cycles 3, 4) instead of 3

Hazard: RAW dependencies 3/4 - with forwarding

Assumption: split-cycle register file *and* MEM→EX forwarding.



- `lw` data is available at the **end** of ME (cycle 4)
- Forwarding wires the ME output directly to `addi`'s EX input
- **1 stall (cycle 4)** — unavoidable: ME ends *after* EX would need to start

Hazard: RAW dependencies 4/4

Consider now the following example:

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆	cycle ₇	cycle ₈
lw \$2, 0(\$4)	IF	ID	EX	ME	WB			
addi \$5, \$2, 10		IF	ID		EX	ME	WB	
add \$12, \$9, \$11			IF		ID	EX	ME	WB

Instruction 3 is independent from the others so we can change the order!

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆	cycle ₇	cycle ₈
lw \$2, 0(\$4)	IF	ID	EX	ME	WB			
add \$12, \$9, \$11		IF	ID	EX	ME	WB		
addi \$5, \$2, 10			IF	ID	EX	ME	WB	

Hazard: WAW dependencies

Two instructions write in the same register!

Consider the following example:

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆
addi \$5, \$11, 42	IF	ID	EX	ME	WB	
addi \$5, \$2, 10		IF	ID	EX	ME	WB

WAW does not produce stalls !
(even when writing in the same memory address)

Hazard: WAR dependencies

One instruction writes where a previous one reads!

Consider the following example:

	cycle ₁	cycle ₂	cycle ₃	cycle ₄	cycle ₅	cycle ₆
addi \$5, \$11, 42	IF	ID	EX	ME	WB	
addi \$11, \$2, 10		IF	ID	EX	ME	WB

WAR do not produce stalls !

Preserving and computing dependencies?

We construct a directed acyclic graph (DAG) to represent the dependencies between instructions:

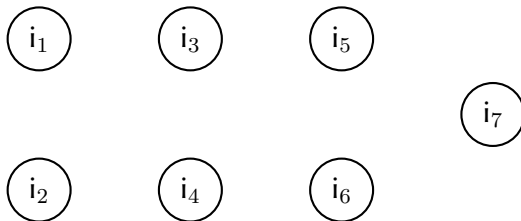
- For each instruction in the basic block, create a corresponding vertex in the graph
- For each dependency between two instructions, create a corresponding (annotated) edge in the graph. Note that this edge is annotated.

Computing the dependency graph

i_1 :	lw	\$1, 0 (\$10)	i_5 :	lw	\$4, 8 (\$10)
i_2 :	lw	\$2, 4 (\$10)	i_6 :	add	\$3, \$1, \$4
i_3 :	add	\$3, \$1, \$2	i_7 :	sw	\$3, 16 (\$10)
i_4 :	sw	\$3, 12 (\$10)			

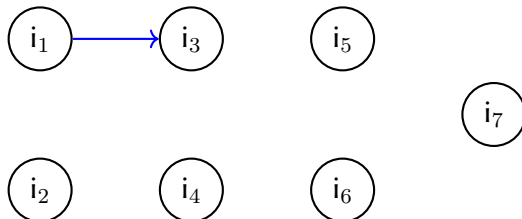
Computing the dependency graph

i_1 : lw \$1, 0 (\$10)	i_5 : lw \$4, 8 (\$10)
i_2 : lw \$2, 4 (\$10)	i_6 : add \$3, \$1, \$4
i_3 : add \$3, \$1, \$2	i_7 : sw \$3, 16 (\$10)
i_4 : sw \$3, 12 (\$10)	



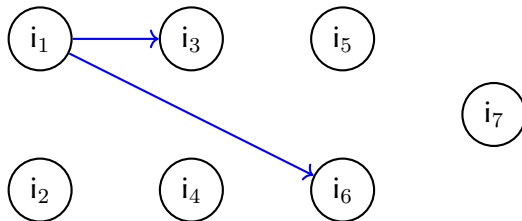
Computing the dependency graph

i_1 : lw \$1, 0 (\$10)	i_5 : lw \$4, 8 (\$10)
i_2 : lw \$2, 4 (\$10)	i_6 : add \$3, \$1, \$4
i_3 : add \$3, \$1, \$2	i_7 : sw \$3, 16 (\$10)
i_4 : sw \$3, 12 (\$10)	



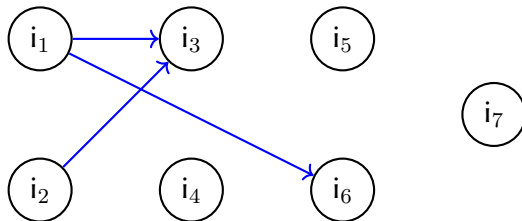
Computing the dependency graph

i_1 : lw \$1, 0(\$10)	i_5 : lw \$4, 8(\$10)
i_2 : lw \$2, 4(\$10)	i_6 : add \$3, \$1, \$4
i_3 : add \$3, \$1, \$2	i_7 : sw \$3, 16(\$10)
i_4 : sw \$3, 12(\$10)	



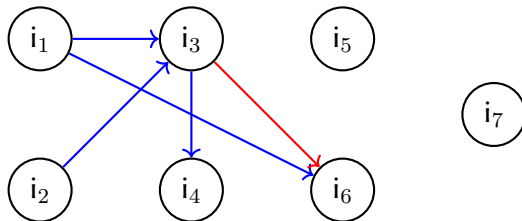
Computing the dependency graph

i_1 : lw \$1, 0(\$10)	i_5 : lw \$4, 8(\$10)
i_2 : lw \$2, 4(\$10)	i_6 : add \$3, \$1, \$4
i_3 : add \$3, \$1, \$2	i_7 : sw \$3, 16(\$10)
i_4 : sw \$3, 12(\$10)	



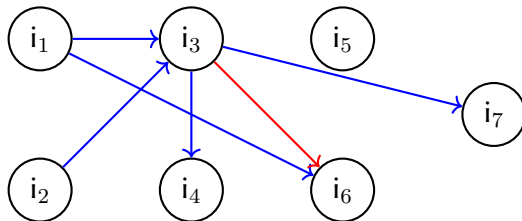
Computing the dependency graph

i_1 : lw	\$1, 0 (\$10)	i_5 : lw	\$4, 8 (\$10)
i_2 : lw	\$2, 4 (\$10)	i_6 : add	\$3, \$1, \$4
i_3 : add	\$3, \$1, \$2	i_7 : sw	\$3, 16 (\$10)
i_4 : sw	\$3, 12 (\$10)		



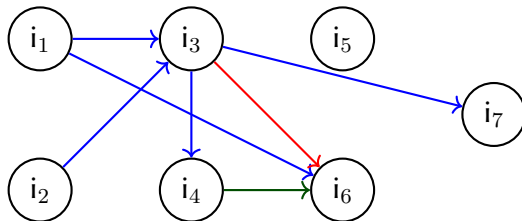
Computing the dependency graph

i_1 : lw	\$1, 0 (\$10)	i_5 : lw	\$4, 8 (\$10)
i_2 : lw	\$2, 4 (\$10)	i_6 : add	\$3, \$1, \$4
i_3 : add	\$3, \$1, \$2	i_7 : sw	\$3, 16 (\$10)
i_4 : sw	\$3, 12 (\$10)		



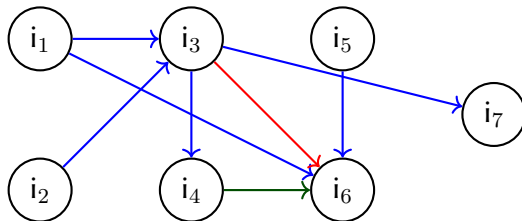
Computing the dependency graph

i_1 : lw	\$1, 0 (\$10)	i_5 : lw	\$4, 8 (\$10)
i_2 : lw	\$2, 4 (\$10)	i_6 : add	\$3, \$1, \$4
i_3 : add	\$3, \$1, \$2	i_7 : sw	\$3, 16 (\$10)
i_4 : sw	\$3, 12 (\$10)		



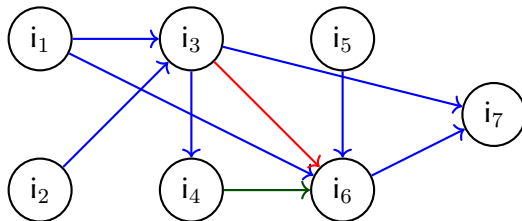
Computing the dependency graph

i_1 : lw	\$1, 0 (\$10)	i_5 : lw	\$4, 8 (\$10)
i_2 : lw	\$2, 4 (\$10)	i_6 : add	\$3, \$1, \$4
i_3 : add	\$3, \$1, \$2	i_7 : sw	\$3, 16 (\$10)
i_4 : sw	\$3, 12 (\$10)		



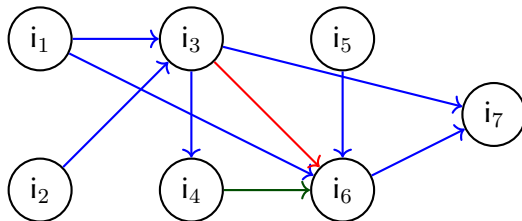
Computing the dependency graph

i_1 :	lw	\$1, 0 (\$10)	i_5 :	lw	\$4, 8 (\$10)
i_2 :	lw	\$2, 4 (\$10)	i_6 :	add	\$3, \$1, \$4
i_3 :	add	\$3, \$1, \$2	i_7 :	sw	\$3, 16 (\$10)
i_4 :	sw	\$3, 12 (\$10)			



Computing the dependency graph

i_1 : lw \$1, 0(\$10)	i_5 : lw \$4, 8(\$10)
i_2 : lw \$2, 4(\$10)	i_6 : add \$3, \$1, \$4
i_3 : add \$3, \$1, \$2	i_7 : sw \$3, 16(\$10)
i_4 : sw \$3, 12(\$10)	



Type of dependency: RAW, WAW, WAR

Preserving dependencies: Critical Path 1/4

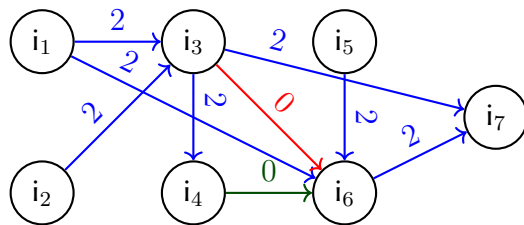
Any (reverse) topological sort of this DAG (i.e. any linear ordering of the vertices which keeps all the edges “pointing forwards”) will maintain the dependencies and hence preserve the correctness of the program.

Preserving dependencies: Critical Path 2/4

The **critical path** represents the longest path between two nodes. We add **delays** (weights) to edges:

- 0 for WAW and WAR dependencies
 - 2 for RAW dependencies with memory access
 - 1 for other RAW dependencies
-
- It determines the minimum execution time assuming ideal resources (perfect parallelism except for dependencies).
 - focus on keeping critical-path instructions from being delayed, as any delay on the critical path increases total runtime.

Preserving dependencies: Critical Path 3/4



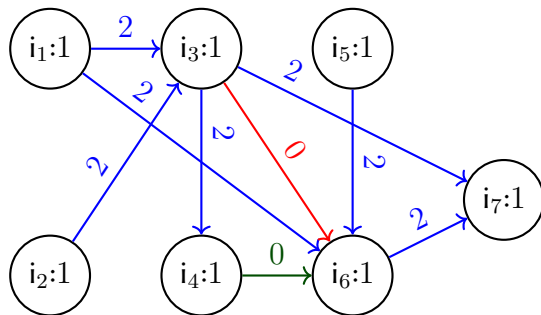
Preserving dependencies: Critical Path 4/4

Algorithm:

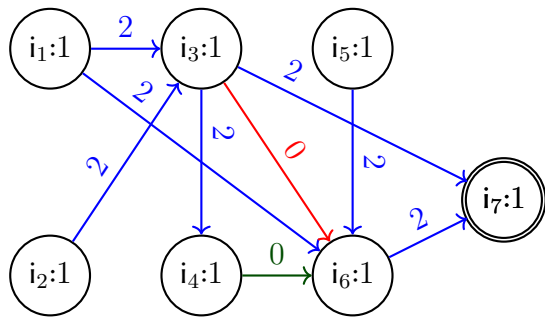
- Associate a weight 1 to each “instruction node”
- For all nodes n_i in topological postorder
 - ▶ If n_i is not a leaf
 - ★ For all nodes n_j in $\text{succ}(n_i)$ do
$$n_i.\text{weight} \leftarrow \max(n_i.\text{weight}, n_j.\text{weight} + \text{delay}(n_i, n_j))$$

Remember “important” edges during computations, they will form the critical path.

Computing the critical path

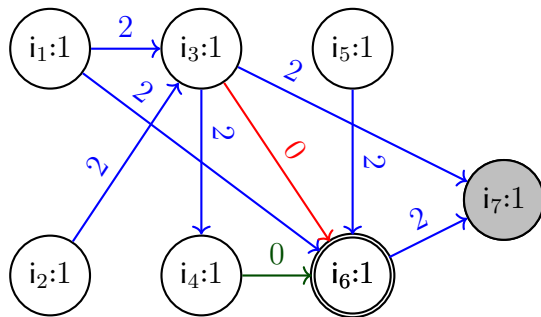


Computing the critical path



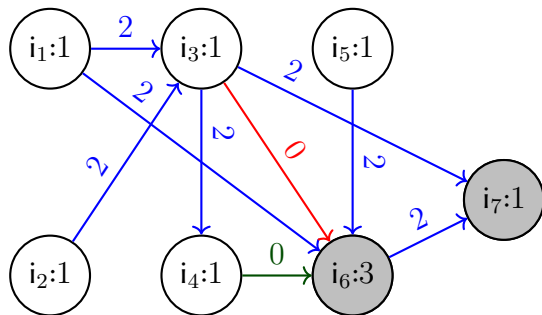
i_7 doesn't have successors, skip it!

Computing the critical path

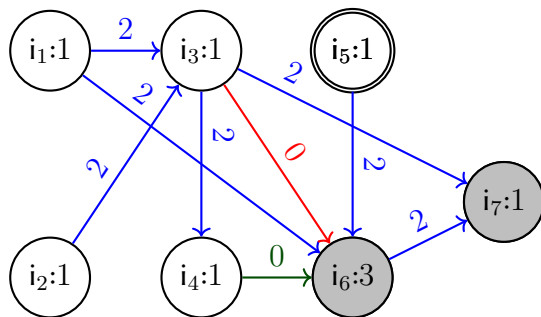


$(i_7.\text{weight} + \text{delay}(i_6, i_7)) = 3 > 1 \implies \text{change } i_6 \text{ weight and mark } i_6 \rightarrow i_7$

Computing the critical path

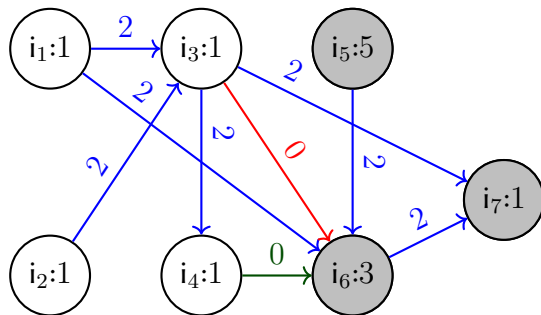


Computing the critical path

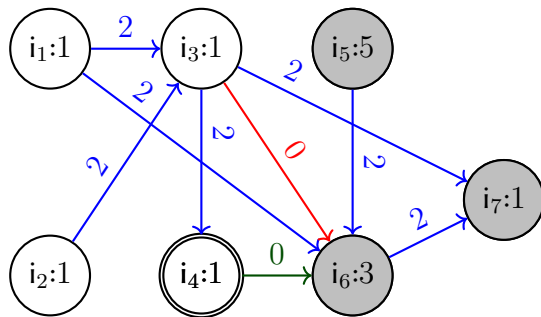


$(i_6.\text{weight} + \text{delay}(i_5, i_6)) = 5 > 1 \implies \text{change } i_5 \text{ weight and mark } i_5 \rightarrow i_6$

Computing the critical path

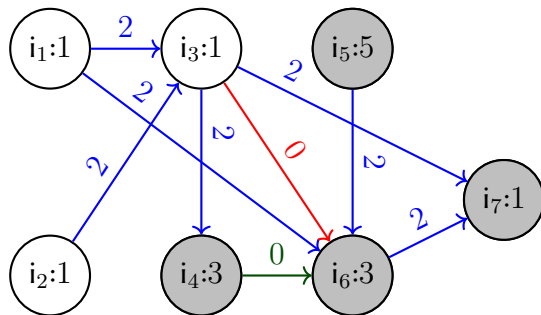


Computing the critical path

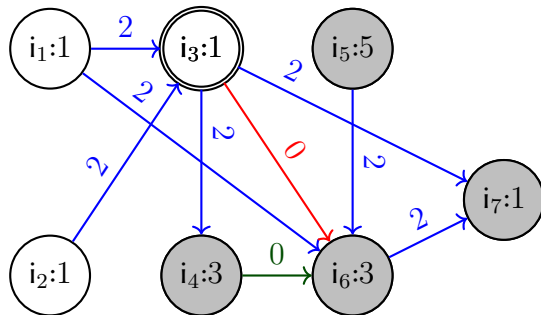


$(i_6.\text{weight} + 0) = 3 > 1 \implies$ change i_4 weight and mark $i_4 \rightarrow i_6$

Computing the critical path

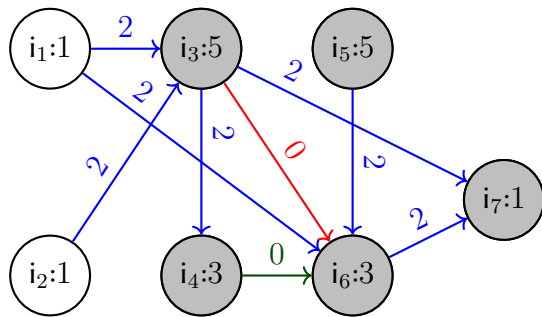


Computing the critical path

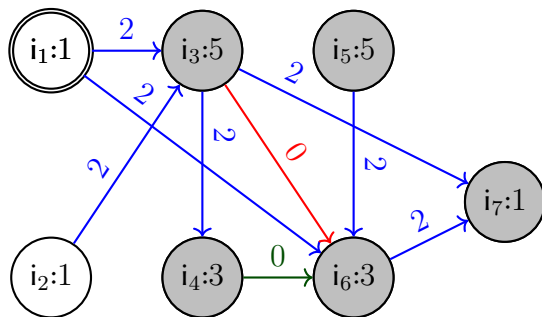


$(i_4.\text{weight} + 2) = 5 > 1 \implies$ change i_3 weight and mark $i_3 \rightarrow i_4$

Computing the critical path

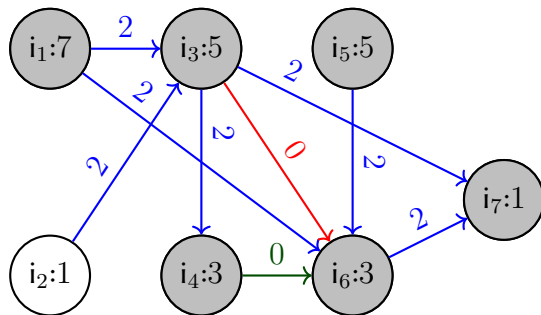


Computing the critical path

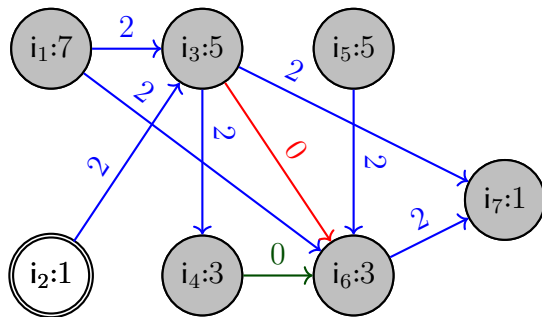


$(i_3.\text{weight} + 2) = 7 > 1 \implies$ change i_1 weight and mark $i_1 \rightarrow i_3$

Computing the critical path

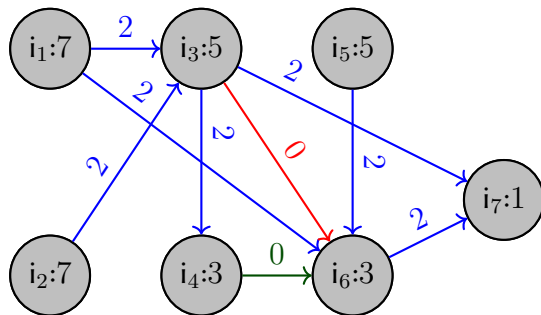


Computing the critical path

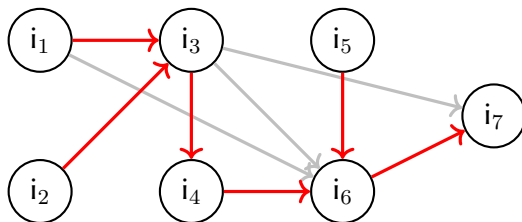


$(i_3.\text{weight} + 2) = 7 > 1 \implies$ change i_2 weight and mark $i_2 \rightarrow i_3$

Computing the critical path



One critical path but many orders



$i_1, i_2, i_3, i_4, i_5, i_6, i_7$
 $i_2, i_1, i_5, i_3, i_4, i_6, i_7$

$i_1, i_2, i_3, i_5, i_4, i_6, i_7$
 $i_1, i_5, i_2, i_3, i_4, i_6, i_7$

$i_2, i_1, i_3, i_5, i_4, i_6, i_7$
 $i_2, i_5, i_1, i_3, i_4, i_6, i_7$

$i_2, i_1, i_3, i_4, i_5, i_6, i_7$
 $i_5, i_1, i_2, i_3, i_4, i_6, i_7$

$i_1, i_2, i_5, i_3, i_4, i_6, i_7$
 $i_5, i_2, i_1, i_3, i_4, i_6, i_7$

All these permutations respect dependencies
but is there a best instruction scheduling?

Performances and Pipeline

Not all orders are equivalent!

- Some dependencies bring hazards that slow down the performances
- Hazard occurs when:
 - ▶ 1 instruction requires the previous instruction has finished
 - ▶ 2 instructions need the same data at the same time: one of the two is blocked

Back to the example – without scheduling

i_1 : lw	\$1, 0 (\$10)	i_5 : lw	\$4, 8 (\$10)
i_2 : lw	\$2, 4 (\$10)	i_6 : add	\$3, \$1, \$4
i_3 : add	\$3, \$1, \$2	i_7 : sw	\$3, 16 (\$10)
i_4 : sw	\$3, 12 (\$10)		

	c ₁	c ₂	c ₃	c ₄	c ₅	c ₆	c ₇	c ₈	c ₉	c ₁₀	c ₁₁	c ₁₂	c ₁₃
i_1	IF	ID	EX	ME	WB								
i_2		IF	ID	EX	ME	WB							
i_3			IF	ID		EX	ME	WB					
i_4				IF		ID	EX	ME	WB				
i_5						IF	ID	EX	ME	WB			
i_6							IF	ID		EX	ME	WB	
i_7								IF		ID	EX	ME	WB

Without scheduling: 2 dependencies, 2 stalls, 13 cycles!

Minimizing Stalls (1/2)

Each time we emit the next instruction, we should try to choose one which

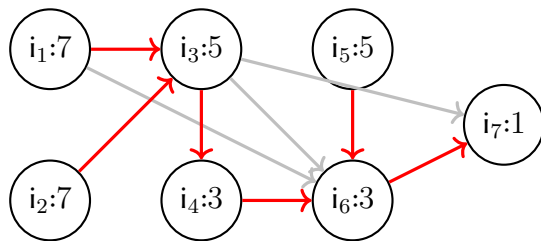
- P_1 does not conflict with the previous emitted instruction
- P_2 : is the most likely to conflict when it's the first of a dependency pair (e.g. prefer `lw` to `add`)
- P_3 : is as far away as possible (along paths in the DAG) from an instruction which can validly be scheduled last

Minimizing Stalls (2/2)

Algorithm:

- Compute the dependency graph and its weights
- Put each “minimal” node (instructions that have no dependencies) in a “candidates” list
- While the list of candidate instructions is not empty
 - ▶ If one instruction satisfies P_1 , P_2 , and P_3 : remove it from the list and emit it.
 - ★ Remove the instruction from the DAG and insert the newly minimal elements into the candidate list.
 - ▶ Otherwise emit a `nop` instruction

Applying scheduling algorithm to the example - 1

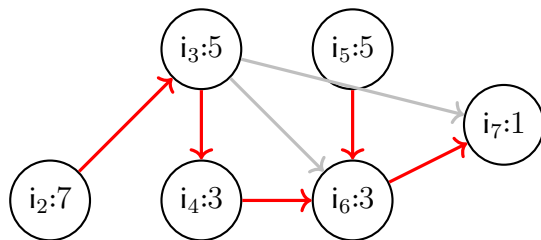


Candidates = $\{i_1, i_2, i_5\}$

Final Order =

Choose i_1 (or i_2) since it satisfies P_1 , P_2 and P_3

Applying scheduling algorithm to the example - 1

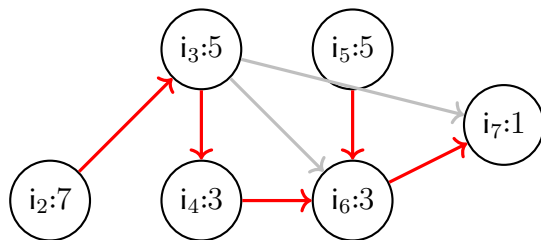


Candidates = $\{i_1, i_2, i_5\}$

Final Order = i_1

Choose i_1 (or i_2) since it satisfies P_1 , P_2 and P_3

Applying scheduling algorithm to the example - 2

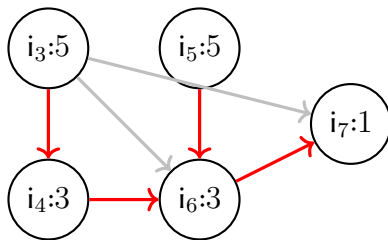


Candidates = $\{i_2, i_5\}$

Final Order = i_1

Choose i_2 since it satisfies P_1 , P_2 and P_3

Applying scheduling algorithm to the example - 2

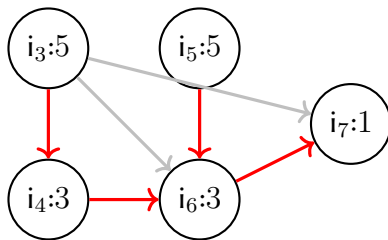


Candidates = $\{i_2, i_5\}$

Final Order = i_1, i_2

Choose i_2 since it satisfies P_1 , P_2 and P_3

Applying scheduling algorithm to the example - 3

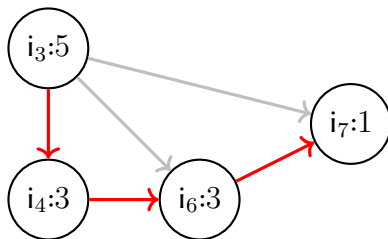


Candidates = $\{i_5, i_3\}$

Final Order = i_1, i_2

Choose i_5 since it satisfies P_1 , P_2 and P_3

Applying scheduling algorithm to the example - 3

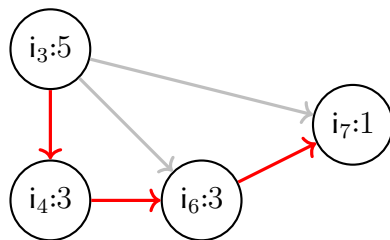


Candidates = $\{i_5, i_3\}$

Final Order = i_1, i_2, i_5

Choose i_5 since it satisfies P_1, P_2 and P_3

Applying scheduling algorithm to the example - 4

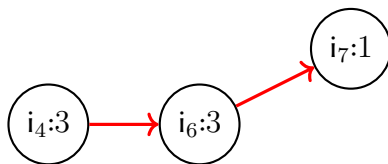


Candidates = $\{i_3\}$

Final Order = i_1, i_2, i_5

Choose i_3 since it's the only candidate

Applying scheduling algorithm to the example - 4

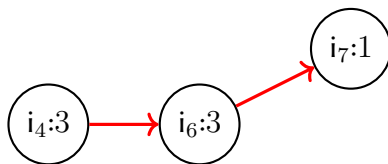


Candidates = $\{i_3\}$

Final Order = i_1, i_2, i_5, i_3

Choose i_3 since it's the only candidate

Applying scheduling algorithm to the example - 5

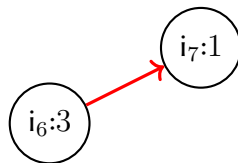


Candidates = $\{i_4\}$

Final Order = i_1, i_2, i_5, i_3

Choose i_4 since it's the only candidate

Applying scheduling algorithm to the example - 5

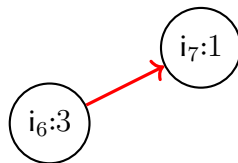


Candidates = $\{i_4\}$

Final Order = i_1, i_2, i_5, i_3, i_4

Choose i_4 since it's the only candidate

Applying scheduling algorithm to the example - 6



Candidates = $\{i_6\}$

Final Order = i_1, i_2, i_5, i_3, i_4

Choose i_6 since it's the only candidate

Applying scheduling algorithm to the example - 6



Candidates = $\{i_6\}$

Final Order = $i_1, i_2, i_5, i_3, i_4, i_6$

Choose i_6 since it's the only candidate

Applying scheduling algorithm to the example - 7



Candidates = $\{i_7\}$

Final Order = $i_1, i_2, i_5, i_3, i_4, i_6$

Choose i_7 since it's the only candidate

Applying scheduling algorithm to the example - 7

Candidates = $\{i_7\}$

Final Order = $i_1, i_2, i_5, i_3, i_4, i_6, i_7$

Choose i_7 since it's the only candidate

Applying scheduling algorithm to the example

Final Order = $i_1, i_2, i_5, i_3, i_4, i_6, i_7$

i_1 :	lw	\$1, 0 (\$10)	i_4 :	sw	\$3, 12 (\$10)
i_2 :	lw	\$2, 4 (\$10)	i_6 :	add	\$3, \$1, \$4
i_5 :	lw	\$4, 8 (\$10)	i_7 :	sw	\$3, 16 (\$10)
i_3 :	add	\$3, \$1, \$2			

	c1	c2	c3	c4	c5	c6	c7	c8	c9	c10	c11
i_1	IF	ID	EX	ME	WB						
i_2		IF	ID	EX	ME	WB					
i_5			IF	ID	EX	ME	WB				
i_3				IF	ID	EX	ME	WB			
i_4					IF	ID	EX	ME	WB		
i_6						IF	ID	EX	ME	WB	
i_7							IF	ID	EX	ME	WB

With scheduling: still 2 dependencies but 0 stalls and 11 cycles!

A word on scheduling strategies

- Sometimes we cannot avoid some stalls
- Computing the critical path can be smarter
- Computing the DAG of dependencies can be done in $O(n^2)$ by scanning backwards through the basic block and adding edges as dependencies arise

A word on performances

We can statically compute the “instructions per cycle” indicator: $IPC = \frac{\text{nb instructions}}{\text{nb cycles}}$, to evaluate compare the results of different rescheduling.

In the previous example:

- before rescheduling: $IPC = \frac{7}{13} = 0.53$
- after rescheduling: $IPC = \frac{7}{11} = 0.63$ (better!)

Table of Contents

- 1 Introduction
- 2 Data dependencies
- 3 Instruction dependencies**

Exercise: Guess the output

```
.set noreorder
.global _start
_start:
    li $a0, 4
    li $v0, 1 # syscall code for printing the integer in $a0
    syscall

    j target
    li $a0, 2

target:

    li $v0, 1
    syscall
```

Why 42?

Problem

When the CPU encounters a branch, it has **already fetched** the next instruction into the pipeline before knowing where to jump.

- The CPU cannot “cancel” this instruction: it **will always be executed**
- Two possible solutions:
 - ▶ **Hardware approach:** insert stalls automatically (1–3 cycles lost)
 - ▶ **Software approach:** expose the problem to the compiler

Assembler Reordering

Removing the directive `.set noreorder` enables reordering

```
.global _start
```

```
_start:
```

```
    li $a0, 1
```

```
    li $v0, 1
```

```
    syscall
```

```
    j target
```

```
    li $a0, 2
```

```
target:
```

```
    li $v0, 1
```

```
    syscall
```

```
.global _start
```

```
_start:
```

```
    li $a0, 1
```

```
    li $v0, 1
```

```
    syscall
```

```
    j target
```

```
    nop      # added by assembler
```

```
    li $a0, 2
```

```
target:
```

```
    li $v0, 1
```

```
    syscall
```

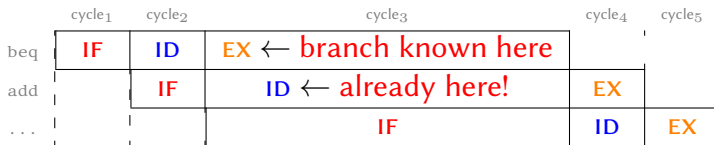
→

Why is the Next Instruction Always Executed?

Physical reality

The pipeline is an **electronic circuit**: at every clock tick, every stage advances. There is no “undo”.

- At each clock edge, every pipeline register is **unconditionally updated**
- Once a signal has propagated through a stage, it cannot be “un-propagated”
- The instruction after a branch is **already decoded** before the branch outcome is known



The Branch Delay Slot

Definition

The instruction **immediately after** a branch (beq, bne, j, jal...) is **always executed**, even if the branch is taken.

```
beq $1, $2, label    ← branch
add $3, $4, $5        ← delay slot: always executed
...                  ← only if branch not taken
label: ...            ← only if branch taken
```

Filling the Delay Slot: 3 Strategies

- 1 Move an independent instruction up (ideal, 0 cycle lost)

<i>Before</i>		<i>After</i>	
add \$3, \$4, \$5		beq \$1, \$2, label	
beq \$1, \$2, label		add \$3, \$4, \$5 ← slot	

- 2 Duplicate the first instruction from the target (acceptable, code size increases slightly)
- 3 Insert a **nop** (last resort, 1 cycle lost)

Link with Instruction Scheduling

After emitting a branch, the scheduling algorithm must find a candidate instruction that can **legally** fill the delay slot.

```
lw $1, 0($2)
lw $3, 4($2)
add $5, $1, $3
beq $4, $0, end
nop      ← 1 stall
```

→

```
lw $1, 0($2)
lw $3, 4($2)
beq $4, $0, end
add $5, $1, $3 ← delay slot filled!
```

Legality Conditions for Filling the Slot

An instruction I can be placed in the delay slot of branch B iff:

- 1 I appears **before** B in the original order
- 2 I is **independent** from B (no RAW, WAR, or WAW dependency with B)
- 3 I is **valid on both paths** (taken and not taken): no dangerous side effects

In practice

After emitting B , the algorithm scans the candidate list for an instruction satisfying all three conditions. Otherwise it emits a `nop`.

Two Responses to This Problem

Using reorder (default)

Flush the pipeline: force all stages after the branch to execute effective nops by overwriting pipeline registers with zeros.

- ✓ Transparent to the programmer
- ✓ Correct always
- ✗ Still loses at least 1 cycle (flush arrives too late for the first slot)

noreorder: take charge

Expose the slot: since the next instruction will always execute anyway, tell the compiler and let it take advantage.

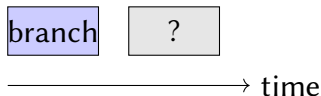
- ✓ 0 cycles lost if compiler fills the slot
- ✗ Compiler must handle it
- ✗ Not transparent (MIPS-specific)

Table of Contents

- 1 Introduction
- 2 Data dependencies
- 3 Instruction dependencies**
 - **Branch Prediction**
 - Loop Unrolling

Why Branch Prediction?

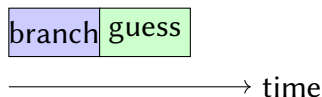
- CPUs use **pipelines** to overlap instructions (assembly-line execution)
- **Branches** break this flow:
 - ▶ Next instruction depends on a condition
 - ▶ CPU does not know which path to fetch early
- This causes a **pipeline stall**:



- Deep pipelines $\Rightarrow$ high penalty (many lost cycles)

Idea: Branch Prediction

- Instead of waiting, CPU **guesses** the branch outcome
- Starts executing instructions **speculatively**
- If prediction is correct:
 - ▶ Pipeline stays full $\Rightarrow$ high performance
- If prediction is wrong:
 - ▶ Discard wrong work (flush pipeline)
 - ▶ Restart from correct path (penalty)



Performance Impact

- Example model:
 - ▶ 20% branches, 80% other instructions
 - ▶ Correct prediction: 1 cycle
 - ▶ Misprediction: ~ 20 cycles
- Without prediction: [≈ 4.8 cycles / instruction]
- With perfect prediction: [≈ 1 cycle / instruction]
- $\Rightarrow$ up to **$5\times$ speedup**
- Even simple predictors (e.g., always taken) give large gains

Simple Branch Prediction Heuristics

- Static (no history)
 - ▶ Always predict **taken** ($\sim 70\%$ accuracy)
 - ▶ **BTFNT**: Backward Taken, Forward Not Taken
 - ★ Loops $\Rightarrow$ backward $\Rightarrow$ taken
- Dynamic (uses history)
 - ▶ 1-bit predictor: last outcome
 - ▶ 2-bit saturating counter: more stable

State-of-the-art hybrid predictors (like TAGE, perceptron-based, or complex gshare variants) achieve around 95–99% accuracy on integer workloads.

Table of Contents

- 1 Introduction
- 2 Data dependencies
- 3 Instruction dependencies**
 - Branch Prediction
 - **Loop Unrolling**

Can we do better?

Consider the following basic block:

i ₁ :	Loop:	lw	\$t0, 0(\$s1)	# t0=array element
i ₂ :		addu	\$t0, \$t0, \$s2	# add scalar from s2
i ₃ :		sw	\$t0, 0(\$s1)	# store result
i ₄ :		addi	\$s1, \$s1, -4	# decrement pointer
i ₅ :		bne	\$s1, \$0, Loop	# branch s1!=0

Can we do better?

Consider the following basic block:

```
i1:  Loop:  lw      $t0, 0($s1)      # t0=array element
i2:      addu    $t0, $t0, $s2      # add scalar from s2
i3:      sw      $t0, 0($s1)      # store result
i4:      addi    $s1, $s1, -4      # decrement pointer
i5:      bne     $s1, $0, Loop      # branch s1!=0
```

	c ₁	c ₂	c ₃	c ₄	c ₅	c ₆	c ₇	c ₈	c ₉	c ₁₀	c ₁₁	c ₁₂	c ₁₃	c ₁₄	c ₁₅	c ₁₆
i ₁	IF	ID	EX	ME	WB											
i ₂		IF	ID		EX	ME	WB									
i ₃			IF					ID	EX	ME	WB					
i ₄								IF	ID	EX	ME	WB				
i ₅									IF				ID	EX	ME	WB

Can we do better?

Consider the following basic block:

```
i1:  Loop:  lw      $t0, 0($s1)      # t0=array element
i2:      addu    $t0, $t0, $s2      # add scalar from s2
i3:      sw      $t0, 0($s1)      # store result
i4:      addi    $s1, $s1, -4      # decrement pointer
i5:      bne     $s1, $0, Loop      # branch s1!=0
```

	c ₁	c ₂	c ₃	c ₄	c ₅	c ₆	c ₇	c ₈	c ₉	c ₁₀	c ₁₁	c ₁₂	c ₁₃	c ₁₄	c ₁₅	c ₁₆
i ₁	IF	ID	EX	ME	WB											
i ₂		IF	ID		EX	ME	WB									
i ₃			IF					ID	EX	ME	WB					
i ₄								IF	ID	EX	ME	WB				
i ₅									IF				ID	EX	ME	WB

16 cycles for 5 instructions that are all dependant (IPC = 0.31)!

Loop Unrolling

- Replicate loop body to expose more parallelism
- Reduces loop-control overhead

At a high level, it can be seen as following:

Without Loop Unrolling

```
int i;  
int sum = 0;  
for (i=0; x < 100; i+=1){  
    sum += tab[i];  
  
}
```

With Loop Unrolling

```
int i;  
int sum=0;  
for (i=0; x < 97; i += 3) {  
    sum = tab[i]    ;  
    sum = tab[i+1];  
    sum = tab[i+2];  
  
}
```

Reminding Iterations

Special care must be taken for pre and post loops operations

```
int i;  
int sum=0;  
for (i=0; x < 97; i += 3) {  
    sum = tab[i] ;  
    sum = tab[i+1];  
    sum = tab[i+2];  
}  
  
// Cleanup loop (remainder)  
for (; i < 100; i++) {  
    sum += tab[i];  
}
```

Pointer Alignment

Modern CPUs load memory in cache lines (typically 64 bytes):

- If data is aligned → fits in one cache line (1 load)
- If not → may span two cache lines (2 loads)

Modern CPUs use SIMD (**S**ingle **I**nstruction **M**ultiple **D**ata):

- If data is aligned 1 load actually loads several value (typically 4)
- Historically, misaligned use → crash (SIGSEGV) or exception

Alignment depends on the alignment of the base address, not on the index being 0.

Pointer Alignment

```
// Prelude: align pointer
```

```
for (; i < 100 && ((uintptr_t)&tab[i] % 16 != 0); i++) {  
    sum+=tab[i];  
}
```

```
// Main unrolled/vectorized loop
```

```
for (; i <= 96; i += 4) {  
    sum += tab[i]    ;  
    sum += tab[i+1];  
    sum += tab[i+2];  
    sum += tab[i+3];  
}
```


Typical Unrolled Loop

```
int i = 0;
// Prelude (alignment / setup)
for (; i < 100 && i % 4 != 0; i++) {
    sum += tab[i];
}
// Main unrolled loop
for (; i <= 96; i += 4) {
    sum += tab[i]    ;
    sum += tab[i+1];
    sum += tab[i+2];
    sum += tab[i+3];
}
// Epilogue (remainder)
for (; i < 100; i++) {
    sum += tab[i];
}
```

Loop Unrolling – back to the example

```
i1:  Loop:  lw      $t0, 0($s1)
i2:          addu   $t0, $t0, $s2
i3:          sw     $t0, 0($s1)
i4:          addi   $s1, $s1, -4
i5:          bne    $s1, $0, Loop
i6:  Loop:  lw      $t0, 0($s1)
i7:          addu   $t0, $t0, $s2
i8:          sw     $t0, 0($s1)
i9:          addi   $s1, $s1, -4
i10:         bne    $s1, $0, Loop
i11: Loop:  lw      $t0, 0($s1)
i12:         addu   $t0, $t0, $s2
i13:         sw     $t0, 0($s1)
i14:         addi   $s1, $s1, -4
i15:         bne    $s1, $0, Loop
```

First duplicate N times the the body of the loop!

Loop Unrolling – back to the example

```
i1:  Loop:  lw      $t0, 0($s1)
i2:          addu   $t0, $t0, $s2
i3:          sw     $t0, 0($s1)
i4:          addi   $s1, $s1, -4
i5:          bne    $s1, $0, Loop
i6:  Loop:  lw      $t0, 0($s1)
i7:          addu   $t0, $t0, $s2
i8:          sw     $t0, 0($s1)
i9:          addi   $s1, $s1, -4
i10:         bne    $s1, $0, Loop
i11: Loop:  lw      $t0, 0($s1)
i12:         addu   $t0, $t0, $s2
i13:         sw     $t0, 0($s1)
i14:         addi   $s1, $s1, -4
i15:         bne    $s1, $0, Loop
```

Remove redundant labels and jump

Loop Unrolling – back to the example

```
i1:  Loop:  lw      $t0, 0($s1)
i2:      addu    $t0, $t0, $s2
i3:      sw      $t0, 0($s1)
i4:      addi    $s1, $s1, -4
i6:      lw      $t1, 0($s1)
i7:      addu    $t1, $t1, $s2
i8:      sw      $t1, 0($s1)
i9:      addi    $s1, $s1, -4
i11:     lw      $t2, 0($s1)
i12:     addu    $t2, $t2, $s2
i13:     sw      $t2, 0($s1)
i14:     addi    $s1, $s1, -4
i15:     bne     $s1, $0, Loop
```

Introduce new temporaries when possible!

Loop Unrolling – back to the example

```
i4:  Loop:  addi    $s1, $s1, -12
i1:      lw     $t0, 0($s1)
i2:      addu   $t0, $t0, $s2
i3:      sw     $t0, 0($s1)
i6:      lw     $t1, 4($s1)
i7:      addu   $t1, $t1, $s2
i8:      sw     $t1, 4($s1)
i11:     lw     $t2, 8($s1)
i12:     addu   $t2, $t2, $s2
i13:     sw     $t2, 8($s1)
i15:     bne    $s1, $0, Loop
```

Grab redundant operation and merge them **carefully!**

Loop Unrolling – back to the example

```
i1:  Loop:  addi    $s1, $s1, -12
i2:      lw     $t0, 0($s1)
i3:      lw     $t1, 4($s1)
i4:      lw     $t2, 8($s1)
i5:      addu   $t0, $t0, $s2
i6:      addu   $t1, $t1, $s2
i7:      addu   $t2, $t2, $s2
i8:      sw     $t0, 0($s1)
i9:      sw     $t1, 4($s1)
i10:     sw     $t2, 8($s1)
i11:     bne    $s1, $0, Loop
```

Schedule the instructions and renumber them

Pros & Cons

Pros:

- We avoid a lot of conditional jumps (and many stalls)
- We now do 19 cycles for 11 instructions: $IPC = 0.57$ (instead of previous 0.31)
- Full loop done after $19 * 34 = 646$ cycles (instead of previous $16 * 100 = 1600$)
- The instruction scheduler works with more **independent** instructions

Cons:

- But we may have to insert a prologue and an epilogue to the loop
- Require more temporaries: register allocation will be harder!

Demo

Take a look at example.c

```
gcc example.c -o example; ./example
```

Try it by yourself in `gcc -funroll-loops`

Instruction Scheduling: Phase Order

Scheduling may be done **before**, **after**, or **both before and after** register allocation.

- **Before register allocation**

- ✓ Maximum parallelism exposed
- ✗ May require more registers than available $\Rightarrow$ introduces **spill/fill code** $\Rightarrow$ hurts performance

- **After register allocation**

- ✓ Also improves placement of spill/fill code
- ✓ Required if architecture has illegal instruction combinations (no interlocks)
- ✗ False dependencies introduced by register allocation **limit instruction motion**

- **Both** (pre- and post-allocation): best of both worlds, used by production compilers (GCC, LLVM)

Summary

